- Consejos para el control del flujo de datos en Java (CAS)
  - Contro del flujo de datos
    - La clase `Scanner`
    - Lectura de datos
      - Problema común a la hora de leer datos
    - Conversión de cadenas a tipos de datos
    - Controlar la entrada de datos con `try-catch`
      - Cómo funciona `try-catch`
    - Control de entrada en un bucle
- Consells per al control del flux de dades en Java (VAL)
  - Control del flux de dades
    - La classe `Scanner`
    - Lectura de dades
      - Problema comú en llegir dades
    - Conversió de cadenes a tipus de dades
    - Controlar l'entrada de dades amb `try-catch`
      - Com funciona `try-catch`
    - Control d'entrada en un bucle

Consejos para el control del flujo de datos en Java (CAS)

Contro del flujo de datos

La clase `Scanner`

La clase `Scanner` permite controlar el flujo de datos en Java de forma sencilla y homogénea. Sirve tanto para la entrada de datos desde consola como para la entrada de datos desde ficheros, empleando los mismos métodos.

Para empezar a usarla, debes importar `Scanner` desde el paquete `java.util`:

```
import java.util.Scanner;
```

No obstante, gracias al funcionamiento de IntelliJ, podemos crear directamente el objeto `Scanner` en nuestro código y el IDE se encargará de hacer la importación. Recomiendo hacerlo de esta forma, ya que es más rápido y más fiable que hacerlo a mano. Al crear el objeto `Scanner`, debemos elegir la fuente desde la que recibiremos el flujo de datos.

Si queremos leer datos desde consola, lo hacemos de la siguiente manera:

```
Scanner scanner = new Scanner(System.in);
```

Si queremos leer datos desde un fichero, lo hacemos de la siguiente manera:

```
import java.io.File;
Scanner scanner = new Scanner(new File("nombre_del_fichero.txt"));
```

A la hora de leer rutas de ficheros, se leen siempre desde la carpeta raíz del proyecto.

Lectura de datos

`Scanner` tiene varios métodos para leer distintos tipos de datos, como:

- **`nextLine()`**: Lee una línea completa como una cadena de texto (`String`), hasta que encuentra un salto de línea. Esto incluye espacios y todo lo que se haya escrito hasta presionar Enter.

```
System.out.println("Ingresa tu nombre:");
String nombre = scanner.nextLine();
System.out.println("Hola, " + nombre);
```

- **`nextInt()`**: Lee el siguiente número entero en la entrada. `nextInt()` solo lee el número y no el salto de línea posterior. Esto puede causar problemas si después de `nextInt()` quieres leer una línea completa usando `nextLine()`.

```
System.out.println("Ingresa tu edad:");
int edad = scanner.nextInt();
System.out.println("Tienes " + edad + " años.");
```

- Aparte de `nextInt()`, contamos con muchos métodos equivalentes para cada tipo de datos primitivo, como `Float`, `Double`, `Boolean`, etc.

Problema común a la hora de leer datos

Un problema común es que al usar `nextInt()` (o cualquier método para leer datos concretos), el salto de línea `\n` que el usuario introduce se queda "atrapado" en el flujo de entrada. Esto causa que se capturen los datos hasta el espacio y sigan estando presentes para la siguiente solicitud de lectura que hagamos, por lo que de una entrada de datos de usuario podríamos rellenar varias solicitudes de `nextInt()`, hasta que una de ellas capturara el salto de línea.

Solución: Para evitar este problema, agregamos una llamada a `nextLine()` después de `nextInt()` para "limpiar" el salto de línea pendiente y todos los datos añadidos después del dato recogido.

```
System.out.println("Ingresa tu edad:");
int edad = scanner.nextInt();
scanner.nextLine(); // Consumir el salto de línea
System.out.println("Ingresa tu nombre:");
String nombre = scanner.nextLine();
```

Otra forma un poco más compleja, y que puede causar más errores, pero que tiene más flexibilidad y control, es usar `nextLine` en combinación con las herramientas para conversión de datos, como veremos a continuación.

Conversión de cadenas a tipos de datos

Java dispone de métodos para la conversión de datos dentro de un tipo de clases de utilidades específicas que se llaman *wrappers*. Los tipos de datos básicos no son objetos, pero podemos usar su objeto equivalente para invocar determinados métodos. Por ejemplo, para

enteros, `int`, el wrapper es `Integer`. Notamos que el *wrapper* siempre tiene el nombre completo empezando en mayúscula. A través de él, accedemos a los métodos estáticos que nos sirven para transformar datos.

Si tienes una entrada como cadena y necesitas convertirla a un tipo numérico, puedes usar métodos como `Integer.parseInt()` para convertir una `String` a `int`. Esto es útil si, por ejemplo, estás leyendo datos de un fichero o de una cadena que no se puede leer directamente como número con `nextInt()`.

```
String numeroEnTexto = "123";
int numero = Integer.parseInt(numeroEnTexto); // Convierte "123" en el
número 123
```

Además de `Integer.parseInt()`, existen otros métodos de *parseo*:

- **`Double.parseDouble()`**: para convertir a `double`.
- **`Float.parseFloat()`**: para convertir a `float`.
- **`Long.parseLong()`**: para convertir a `long`.

Estos métodos son útiles porque puedes controlar mejor el flujo de entrada y, por ejemplo, leer una línea completa con `nextLine()` y luego *parsearla*.

Controlar la entrada de datos con `try-catch`

Al trabajar con entradas de usuario, a menudo es necesario asegurarse de que el usuario introduzca datos válidos. Por ejemplo, si el usuario introduce una letra en lugar de un número y estamos usando `nextInt()`, el programa arrojará una excepción `InputMismatchException`. También tendremos problemas de este tipo al *parsear* los datos recogidos por un `nextLine()`.

Podemos controlar estos errores usando `try-catch` para manejar situaciones en las que el usuario no introduzca el tipo de dato esperado:

```
try {
    System.out.println("Ingresa un número entero:");
    int numero = scanner.nextInt();
    System.out.println("El número ingresado es: " + numero);
} catch (InputMismatchException e) {
    System.out.println("Error: No ingresaste un número entero. Inténtalo de
nuevo.");
    scanner.nextLine(); // Limpiar el buffer para el siguiente intento
}
```

Si no sabemos qué excepción toca, podemos usar la excepción básica, `Exception`, que es la madre de todas las excepciones. En el siguiente ejemplo, vemos el código de arriba pero

usando nextLine para capturar los datos y el mensaje por defecto de la excepción. Además, usamos `System.err` para que nos devuelva el mensaje en forma de error.

```
try {
    System.out.println("Ingresa un número entero:");
    int numero = Integer.parseInt(scanner.nextLine());
    System.out.println("El número ingresado es: " + numero);
} catch (Exception e) {
    System.err.println("Error: " + e.getMessage());

}
```

Cómo funciona `try-catch`

1. **`try`**: Colocamos el código que queremos intentar ejecutar en el bloque `try`. Aquí es donde podría ocurrir una excepción.
2. **`catch`**: Si ocurre una excepción dentro del bloque `try`, el programa salta al bloque `catch`. Aquí podemos manejar el error de manera amigable, como mostrar un mensaje informativo.

El bloque `catch` también se puede personalizar para capturar diferentes tipos de excepciones, si sabes específicamente qué tipo de error podría ocurrir.

A nivel conceptual, funciona como *"Intenta realizar el código de dentro del bloque `try{}`. Si no lo consigues, ejecuta el código dentro del bloque `catch(Exception e)` y gestiona el error que te ha impedido conseguir ejecutar el código"*.

Control de entrada en un bucle

Para hacer que el programa pida repetidamente al usuario que ingrese un valor válido, puedes usar un bucle junto con `try-catch`. Este es un ejemplo donde se le pide al usuario un número entero y no se acepta otro tipo de entrada:

```
int numero;
boolean entradaCorrecta = false;
while (!entradaCorrecta) {
    System.out.println("Por favor, ingresa un número entero:");
    try {
        numero = scanner.nextInt();
        entradaCorrecta = true; // Habilitamos la salida del bucle si la
entrada es válida
    } catch (InputMismatchException e) {
        System.out.println("Entrada inválida. Asegúrate de ingresar un
número entero.");
        scanner.nextLine(); // Limpiar el buffer para el siguiente intento
    }
```

```
}
System.out.println("Número ingresado correctamente: " + numero);
```

Esta es una forma elegante de controlar una entrada de datos, sobre todo de cara a un producto final. Evita que el programa "explote" por algún error en la entrada del usuario. No obstante, para ejercicios y programas en los cuales tenemos una entrada de datos controlada, solemos asumir que siempre la vamos a hacer de forma correcta, para evitar escribir demasiado código y centrarnos más en el problema que queremos resolver que en los problemas que un usuario despistado o con ganas de reventar el sistema nos pudiera ocasionar.

Sin embargo, el control de la entrada de datos es fundamental por motivos de seguridad y para evitar filtraciones de datos, así que no debe ser tomado a la ligera e implementarlo siempre que sea conveniente.

Consells per al control del flux de dades en Java (VAL)

Control del flux de dades

La classe `Scanner`

La classe `Scanner` permet controlar el flux de dades en Java de manera senzilla i homogènia. Serveix tant per a l'entrada de dades des de consola com per a l'entrada de dades des de fitxers, emprant els mateixos mètodes.

Per començar a utilitzar-la, has d'importar `Scanner` des del paquet `java.util`:

```
import java.util.Scanner;
```

No obstant això, gràcies al funcionament d'IntelliJ, podem crear directament l'objecte `Scanner` en el nostre codi, i l'IDE s'encarregarà de fer la importació. Recomane fer-ho d'aquesta manera, ja que és més ràpid i fiable que fer-ho manualment. En crear l'objecte `Scanner`, hem d'elegir la font des de la qual rebrem el flux de dades.

Si volem llegir dades des de la consola, ho fem de la manera següent:

```
Scanner scanner = new Scanner(System.in);
```

Si volem llegir dades des d'un fitxer, ho fem de la manera següent:

```
import java.io.File;
Scanner scanner = new Scanner(new File("nom_del_fitxer.txt"));
```

A l'hora de llegir rutes de fitxers, es llegeixen sempre des de la carpeta arrel del projecte.

Lectura de dades

`Scanner` té diversos mètodes per a llegir diferents tipus de dades, com ara:

- **`nextLine()`**: Llig una línia completa com una cadena de text (`String`), fins que troba un salt de línia. Açò inclou espais i tot el que s'haja escrit fins a prémer Enter.

  ```
  System.out.println("Introdueix el teu nom:");
  String nom = scanner.nextLine();
  System.out.println("Hola, " + nom);
  ```

- **`nextInt()`**: Llig el següent nombre enter en l'entrada. `nextInt()` només llig el nombre i no el salt de línia posterior, la qual cosa pot causar problemes si després de `nextInt()` vols llegir una línia completa usant `nextLine()`.

  ```
  System.out.println("Introdueix la teua edat:");
  int edat = scanner.nextInt();
  System.out.println("Tens " + edat + " anys.");
  ```

- A més de `nextInt()`, tenim molts mètodes equivalents per a cada tipus de dades primitiu, com `Float`, `Double`, `Boolean`, etc.

Problema comú en llegir dades

Un problema comú és que en usar `nextInt()` (o qualsevol mètode per a llegir dades concretes), el salt de línia `\n` que l'usuari introdueix es queda "atrapat" en el flux d'entrada. Açò causa que es capturen les dades fins a l'espai, que es mantenen en el flux per a la següent sol·licitud de lectura, per la qual cosa una sola entrada de dades podria omplir diverses sol·licituds de `nextInt()`, fins que una d'elles capturara el salt de línia.

Solució: Per a evitar aquest problema, afegim una crida a `nextLine()` després de `nextInt()` per a "netejar" el salt de línia pendent i totes les dades afegides després del valor recollit.

```
System.out.println("Introdueix la teua edat:");
int edat = scanner.nextInt();
scanner.nextLine(); // Consumir el salt de línia
```

```
System.out.println("Introdueix el teu nom:");
String nom = scanner.nextLine();
```

Una altra forma un poc més complexa, i que pot causar més errors però que té més flexibilitat i control, és usar `nextLine` en combinació amb les eines per a conversió de dades, com veurem a continuació.

Conversió de cadenes a tipus de dades

Java disposa de mètodes per a la conversió de dades dins d'un tipus de classes d'utilitats específiques anomenades *wrappers*. Els tipus de dades bàsics no són objectes, però podem usar el seu objecte equivalent per a invocar determinats mètodes. Per exemple, per a enters, `int`, el wrapper és `Integer`. Notem que el *wrapper* sempre té el nom complet començant en majúscula. A través d'ell, accedim als mètodes estàtics que ens serveixen per a transformar dades.

Si tens una entrada com una cadena i necessites convertir-la a un tipus numèric, pots usar mètodes com `Integer.parseInt()` per a convertir una `String` a `int`. Açò és útil si, per exemple, estàs llegint dades d'un fitxer o d'una cadena que no es pot llegir directament com a nombre amb `nextInt()`.

```
String numeroEnText = "123";
int numero = Integer.parseInt(numeroEnText); // Converteix "123" en el
nombre 123
```

A més de `Integer.parseInt()`, existeixen altres mètodes de *parseig*:

- **`Double.parseDouble()`**: per a convertir a `double`.
- **`Float.parseFloat()`**: per a convertir a `float`.
- **`Long.parseLong()`**: per a convertir a `long`.

Aquests mètodes són útils perquè permeten controlar millor el flux d'entrada i, per exemple, llegir una línia completa amb `nextLine()` i després *parsejar-la*.

Controlar l'entrada de dades amb `try-catch`

En treballar amb entrades de l'usuari, sovint cal assegurar-se que l'usuari introdueix dades vàlides. Per exemple, si l'usuari introdueix una lletra en lloc d'un nombre i estem usant `nextInt()`, el programa llançarà una excepció `InputMismatchException`. També tindrem problemes d'aquest tipus en *parsejar* les dades recollides per un `nextLine()`.

Podem controlar aquests errors usant `try-catch` per a manejar situacions en què l'usuari no introdueix el tipus de dada esperat:

```
try {
    System.out.println("Introdueix un nombre enter:");
    int numero = scanner.nextInt();
    System.out.println("El nombre introduït és: " + numero);
} catch (InputMismatchException e) {
    System.out.println("Error: No has introduït un nombre enter. Torna-ho a
intentar.");
    scanner.nextLine(); // Netejar el buffer per al següent intent
}
```

Si no sabem quina excepció s'està produint, podem usar l'excepció bàsica, `Exception`, que és la mare de totes les excepcions. En l'exemple següent, veiem el codi de dalt però usant `nextLine` per a capturar les dades i el missatge per defecte de l'excepció. A més, usem `System.err` per a mostrar el missatge en forma d'error.

```
try {
    System.out.println("Introdueix un nombre enter:");
    int numero = Integer.parseInt(scanner.nextLine());
    System.out.println("El nombre introduït és: " + numero);
} catch (Exception e) {
    System.err.println("Error: " + e.getMessage());

}
```

Com funciona `try-catch`

1. **`try`**: Col·loquem el codi que volem intentar executar en el bloc `try`. Ací és on podria ocórrer una excepció.
2. **`catch`**: Si ocorre una excepció dins del bloc `try`, el programa salta al bloc `catch`. Ací podem manejar l'error de manera amigable, com mostrar un missatge informatiu.

El bloc `catch` també es pot personalitzar per a capturar diferents tipus d'excepcions, si sabem específicament quin tipus d'error podria ocórrer.

Conceptualment, funciona com *"Intenta realitzar el codi de dins del bloc `try{}`. Si no ho aconsegueixes, executa el codi dins del bloc `catch(Exception e)` i gestiona l'error que t'ha impedit aconseguir executar el codi"*.

Control d'entrada en un bucle

Per a fer que el programa demane repetidament a l'usuari que introduïsca un valor vàlid, pots usar un bucle junt amb `try-catch`. Aquest és un exemple on se li demana a l'usuari un nombre enter i no s'accepta cap altre tipus d'entrada:

```
int numero;
boolean entradaCorrecta = false;
while (!entradaCorrecta) {
    System.out.println("Per favor, introdueix un nombre enter:");
    try {
        numero = scanner.nextInt();
        entradaCorrecta = true; // Habilitem l'eixida del bucle si
l'entrada és vàlida
    } catch (InputMismatchException e) {
        System.out.println("Entrada no vàlida. Assegura't d'introduir un
nombre enter.");
        scanner.nextLine(); // Netejar el buffer per al següent intent
    }


}
System.out.println("Nombre introduït correctament: " + numero);
```

Aquesta és una manera elegant de controlar una entrada de dades, especialment per a un producte final. Evita que el programa "es trenque" per algun error en l'entrada de l'usuari. No obstant això, per a exercicis i programes en què tenim una entrada de dades controlada, solem assumir que sempre es farà de manera correcta, per evitar escriure massa codi i centrar-nos més en el problema que volem resoldre que en els problemes que un usuari despistat o amb ganes de desestabilitzar el sistema ens poguera ocasionar.

Tanmateix, el control de l'entrada de dades és fonamental per motius de seguretat i per evitar filtracions de dades, així que no s'ha de prendre a la lleugera i cal implementar-lo sempre que siga convenient.